

# CS201 – Data Structures II

## Lecture 03

Dr. Muhammad Mobeen Movania

# Outline

- Array-based List
- Implementation of the backing array
- ArrayStack
- ArrayQueue
- ArrayDeque
- Summary

# Array-based List

- Implementation of List and Queue interface with a ***backing array*** (data will be stored in an array)
- Advantages
  - Constant time access to elements i.e. `get(i)` and `set(i,x)` run in constant time
- Disadvantages
  - Adding or removing an element near the middle of a list is cumbersome
    - a large number of elements in the array need to be shifted to make room for the newly added element
    - `add(i,x)` and `remove(i)` have running times that depend on **n** and **i**.
  - Arrays cannot expand and contract
    - A new array is created and all old elements are copied into the new array

# Implementation of the backing array

array

```
T *a;  
int length;
```

The size of an `array` is specified at the time of creation:

array

```
array(int len) {  
    length = len;  
    a = new T[length];  
}
```

The elements of an array can be indexed:

array

```
T& operator[](int i) {  
    assert(i >= 0 && i < length);  
    return a[i];  
}
```

array

```
array<T>& operator=(array<T> &b) {  
    if (a != NULL) delete[] a;  
    a = b.a;  
    b.a = NULL;  
    length = b.length;  
    return *this;  
}
```

# ArrayStack

- Keeps track of the number of elements currently stored in the list through (n)
- Keeps a backing array (a) with more elements than required
  - At all times,  $a.length \geq n$
- The list elements are stored in  $a[0], \dots, a[n - 1]$

```
array<T> a;  
int n;  
int size() {  
    return n;  
}
```

ArrayStack

# ArrayStack – List Interface functions

```
T get(int i) {                                Cost:  $O(1)$   
    return a[i];  
}
```

```
T set(int i, T x) {                            Cost:  $O(1)$   
    T y = a[i];  
    a[i] = x;  
    return y;  
}
```

# ArrayStack – List Interface functions

```
void add(int i, T x) {  
    if (n + 1 > a.length)  
        resize();  
    for (int j = n; j > i; j--)  
        a[j] = a[j - 1];  
    a[i] = x;  
    n++;  
}
```

# Example of add operation

|   |   |   |   |  |  |
|---|---|---|---|--|--|
| b | r | e | d |  |  |
|---|---|---|---|--|--|

|   |   |   |   |   |  |
|---|---|---|---|---|--|
| b | r | e | e | d |  |
|---|---|---|---|---|--|

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| b | r | e | e | d | r |
|---|---|---|---|---|---|

|   |   |   |   |   |   |  |  |  |  |  |  |
|---|---|---|---|---|---|--|--|--|--|--|--|
| b | r | e | e | d | r |  |  |  |  |  |  |
|---|---|---|---|---|---|--|--|--|--|--|--|

|   |   |   |   |   |   |   |  |  |  |  |  |
|---|---|---|---|---|---|---|--|--|--|--|--|
| b | r | e | e | d | e | r |  |  |  |  |  |
|---|---|---|---|---|---|---|--|--|--|--|--|

$\text{add}(2, e)$

$\text{add}(5, r)$

$\text{add}(5, e)^*$



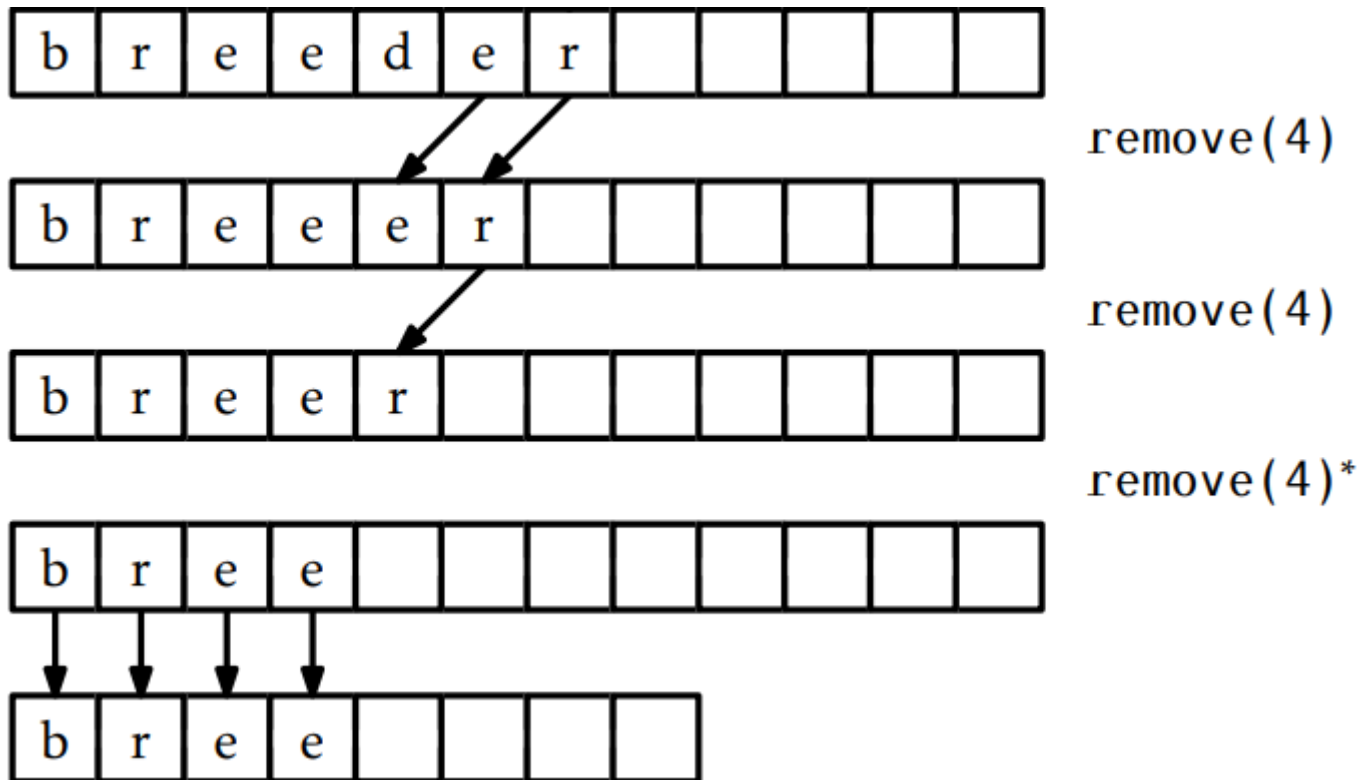
# Cost of add operation

- If we ignore the cost of `resize()` then the cost of add operation is proportional to the number of elements we need to shift  $(n-i)$  to make room for the new element  $x$
- Cost:  $O(n-i)$

# ArrayStack – List Interface functions

```
T remove(int i) {  
    T x = a[i];  
  
    for(int j = i; j < n-1; j++)  
        a[j] = a[j + 1];  
  
    n--;  
  
    if(a.length >= 3 * n)  
        resize();  
  
    return x;  
}
```

# Example of remove operation



# Cost of remove operation

- If we ignore the cost of `resize()` then the cost of remove operation is proportional to the number of elements we shift i.e.  $(n-i)$
- Cost:  $O(n-i)$

# ArrayStack: resize operation

- Helps ArrayStack grow or shrink
- Allocates a new array  $b$  whose size is  $2n$  and copies the  $n$  elements of  $a$  into the first  $n$  positions in  $b$ , and then sets  $a$  to  $b$ .

```
void resize() {  
    array b(max(2 * n, 1));  
    for (int i = 0; i < n; i++)  
        b[i] = a[i];  
    a = b;  
}
```

# Cost of resize operation

- Instead of determining the cost for each add/remove operation, we calculate the cost using **amortized analysis**
  - cost of all calls to `resize()` during a sequence of  $m$  calls to `add(i,x)` or `remove(i)`

**Lemma 2.1.** *If an empty ArrayStack is created and any sequence of  $m \geq 1$  calls to `add(i,x)` and `remove(i)` are performed, then the total time spent during all calls to `resize()` is  $O(m)$ .*

# Proof

- The total calls to add and remove since last resize call are  $n/2-1$
- Assumptions
  - $n_i$  is the value of  $n$  for  $i^{\text{th}}$  call to resize
  - $r$  is the total calls to resize
- Total calls to add or remove are

$$\sum_{i=1}^r (n_i/2 - 1) \leq m$$
$$\sum_{i=1}^r n_i \leq 2m + 2r$$

# Proof

- Total time spent during all resize calls

$$\sum_{i=1}^r O(n_i) \leq O(m + r) = O(m)$$

- No. of calls to `add(i,x)` or `remove(i)` between the  $(i-1)^{\text{th}}$  and the  $i^{\text{th}}$  call to `resize()` is at least  $n_i/2$ .
- Resize is called in two cases:
  - add is called and the backing array is full
    - In the previous call to `resize()`
      - after previous call, the size of `a` was `a.length`, but the number of elements stored in `a` was at most `a.length/2 =  $n_i/2$`
      - Now the size of `a` is  $n_i = \text{a.length}$  so at least  $n_i/2$  calls to `add` have been made since the previous call of `resize`



# Proof

- Resize is called when
  - Remove is called and the backing array is 3x full,  $a.length() \geq 3n = 3n_i$ 
    - After the previous call to `resize()`
      - The total number of elements in `a` was  $a.length/2 - 1$
      - Now `a` contains  $n_i \leq a.length/3$  elements
    - So the total remove calls ( $R$ ) since last resize call are  $a.length/2 - 1 - a.length/3$
    - $R \geq n_i/2 - 1$

# Cost of other operations

- `get(i)` and `set(i,x)` in  $O(1)$  time per operation
- `add(i,x)` and `remove(i)` in  $O(1 + n - i)$  time per operation.
  - $O(1)$  is the cost of element assignment for add
  - $O(n-i)$  is the cost for displacement of  $n-i$  elements

# ArrayQueue

- Implements a FIFO queue
  - Elements are added at one end using add function and removed in the same order (from another end) using remove function
  - One of the two operations (add/remove) works on the head of the list meaning it has a running time proportional to  $n$
- Representation using array
  - Infinite array using modular arithmetic
  - Integer  $j$  stores the index of the element to be removed next
  - Integer  $n$  gives the total number of elements in the queue

```
ArrayQueue  
array<T> a;  
int j;  
int n;
```

# ArrayQueue

- Representation using array
  - elements are stored linearly,
    - $a[j], a[j+1], a[j+2], \dots, a[j+n-1]$
  - May be represented using modular arithmetic
    - $a[j \% a.length], a[(j + 1) \% a.length], \dots, a[(j+n-1) \% a.length]$
  - Initially  $j=0$  and  $n=0$
  - Add
    - places the item at  $a[j+n]$  and increments  $n$
  - Remove
    - removes the item at  $a[j]$  increment  $j$  and decrement  $n$

# ArrayQueue Implementation

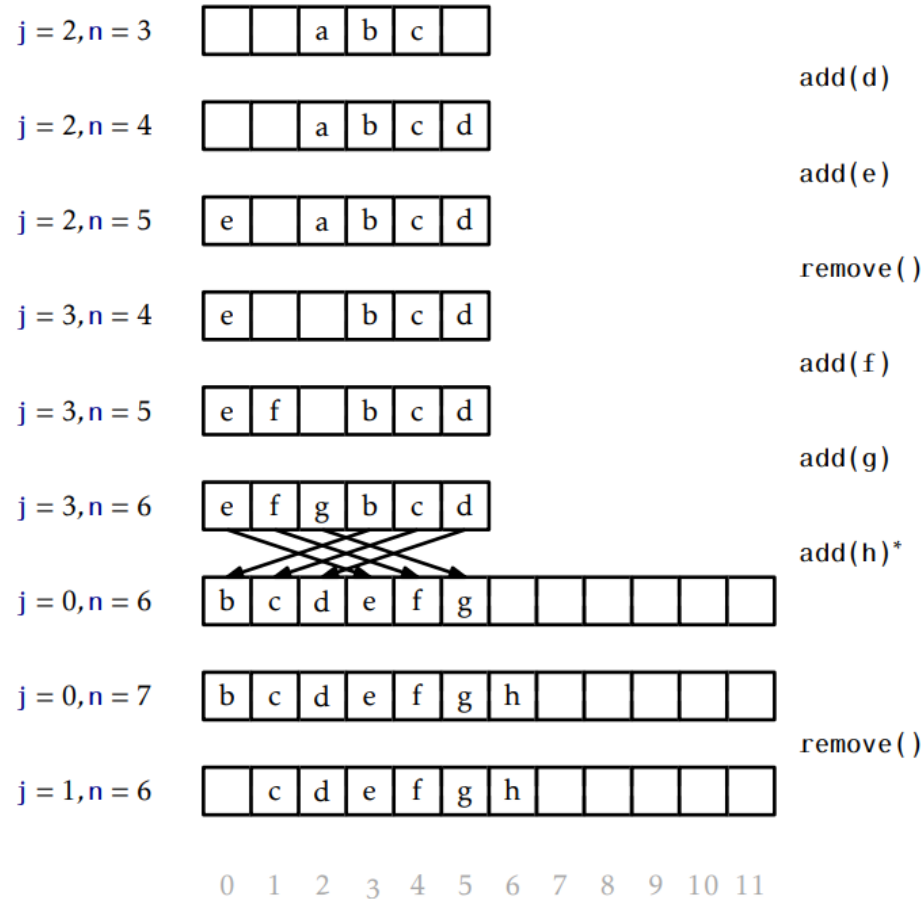
ArrayQueue

```
bool add(T x) {  
    if (n + 1 > a.length) resize();  
    a[(j+n) % a.length] = x;  
    n++;  
    return true;  
}
```

ArrayQueue

```
T remove() {  
    T x = a[j];  
    j = (j + 1) % a.length;  
    n--;  
    if (a.length >= 3*n) resize();  
    return x;  
}
```

# Example: Add/Remove on ArrayQueue



# Resize in ArrayQueue

- Implementation is similar to ArrayStack

ArrayQueue

```
void resize() {  
    array<T> b(max(1, 2*n));  
    for (int k = 0; k < n; k++)  
        b[k] = a[(j+k)%a.length];  
    a = b;  
    j = 0;  
}
```

- 1) Allocates a new array, b, of size  $2n$
- 2) copy  $a[j], a[(j + 1)\%a.length], \dots, a[(j + n - 1)\%a.length]$  onto  $b[0], b[1], \dots, b[n - 1]$  and
- 3) sets  $j = 0$ .

# ArrayQueue – Complexity

- Ignoring the cost of `resize`, `ArrayQueue` supports the operations `add(x)` and `remove()` in  $O(1)$  time per operation
- In an empty `ArrayQueue`, any sequence of  $m$  `add(i,x)` and `remove(i)` operations results in a total of  $O(m)$  time spent during all calls to `resize()`



# ArrayDeque

- It allows for efficient addition and removal at both ends
- It implements the List interface by using the same circular array technique used to represent an ArrayQueue.

# ArrayDeque - Implementation

ArrayDeque

```
array<T> a;  
int j;  
int n;
```

ArrayDeque

```
T get(int i) {  
    return a[(j + i) % a.length];  
}  
T set(int i, T x) {  
    T y = a[(j + i) % a.length];  
    a[(j + i) % a.length] = x;  
    return y;  
}
```

# ArrayDeque – Add function

- Checks the given index  $i$ 
  - If  $i < n/2$ , it moves all elements from  $a[0]$  to  $a[i-1]$  one place to the left
  - Otherwise it moves all elements from  $a[i]$  to  $a[n-1]$  one place to the right
- This guarantees that  $\text{add}(i, x)$  never has to shift more than  $\min\{i, n - i\}$  elements
  - This reduces the running time of  $\text{add}$  (ignoring the cost of a  $\text{resize}$ ) to  $O(1 + \min\{i, n - i\})$ .

# ArrayDeque - Implementation

ArrayDeque

```
void add(int i, T x) {
    if (n + 1 > a.length) resize();
    if (i < n/2) { // shift a[0],...,a[i-1] left one position
        j = (j == 0) ? a.length - 1 : j - 1;
        for (int k = 0; k <= i-1; k++)
            a[(j+k)%a.length] = a[(j+k+1)%a.length];
    } else { // shift a[i],...,a[n-1] right one position
        for (int k = n; k > i; k--)
            a[(j+k)%a.length] = a[(j+k-1)%a.length];
    }
    a[(j+i)%a.length] = x;
    n++;
}
```

# ArrayDeque – Remove function

- Works similar to add function
- If  $i < n/2$ 
  - It shifts elements  $a[0], \dots, a[i - 1]$  right by one position
- Else
  - shifts the elements  $a[i + 1], \dots, a[n - 1]$  left by one position
- Thus, remove never spends more than  $O(1 + \min\{i, n - i\})$  time to shift elements

# ArrayDeque – Implementation

ArrayDeque

```
T remove(int i) {  
    T x = a[(j+i)%a.length];  
    if (i < n/2) { // shift a[0],...,[i-1] right one position  
        for (int k = i; k > 0; k--)  
            a[(j+k)%a.length] = a[(j+k-1)%a.length];  
        j = (j + 1) % a.length;  
    } else { // shift a[i+1],...,a[n-1] left one position  
        for (int k = i; k < n-1; k++)  
            a[(j+k)%a.length] = a[(j+k+1)%a.length];  
    }  
    n--;  
    if (3*n < a.length) resize();  
    return x;  
}
```

# ArrayDeque – Complexity

- Ignoring the cost of resize,
  - ArrayDeque supports the operations `get(i)` and `set(i,x)` in  $O(1)$  time per operation
  - `add(i,x)` and `remove(i)` operations in  $O(1 + \min\{i, n - i\})$  time per operation
- In an empty ArrayDeque, any sequence of  $m$  `add(i,x)` and `remove(i)` operations results in a total of  $O(m)$  time spent during all calls to `resize()`

# Summary

| Data Structure | get(i)/set(i,x) | add(i,x)/remove(i)                               |
|----------------|-----------------|--------------------------------------------------|
| ArrayStack     | $O(1)$          | $O(n-i)$                                         |
| ArrayQueue     | N/A             | $O(1)^*$ or $O(m)$ for $m$ add/remove operations |
| ArrayDeque     | $O(1)$          | $O(\min(i, n-i))$                                |

\* Ignoring resize call



# Summary

- We studied about Array-based Lists
- We talked about
  - ArrayStack which uses the List interface
  - ArrayQueue which uses the FIFO queue interface and
  - ArrayDeque which uses the List interface
- We talked about the operations supported by the three discussed data structures
- We discussed about the time complexity of the different operations

# Book Readings

- ODS: Chapter 2 pages: 29-43

# Next Lecture

- Lecture 04